**SYDE 556/750**

# Simulating Neurobiological Systems
# Lecture 7: Temporal Basis Functions

Chris Eliasmith

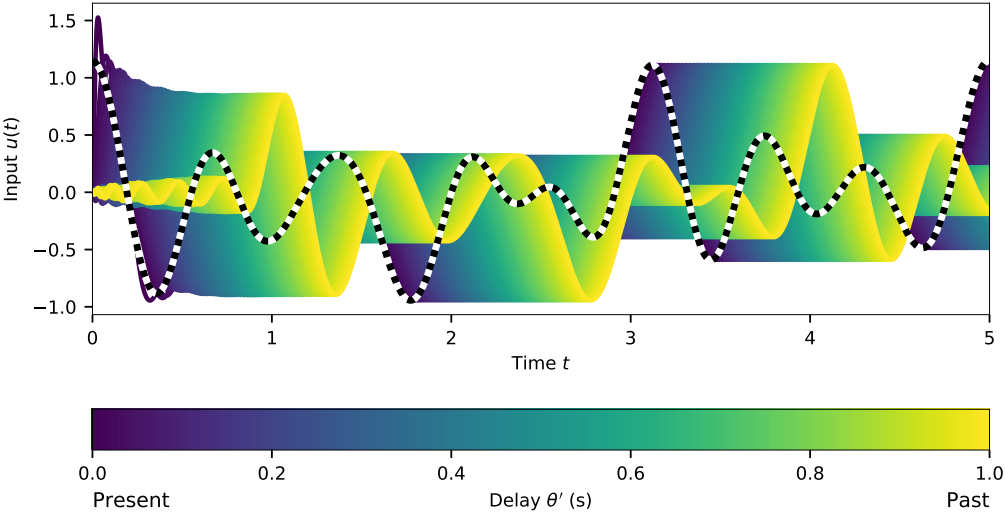
October 21, 2024

- Slide design: Andreas Stöckel
- Content: Terry Stewart, Andreas Stöckel, Chris Eliasmith

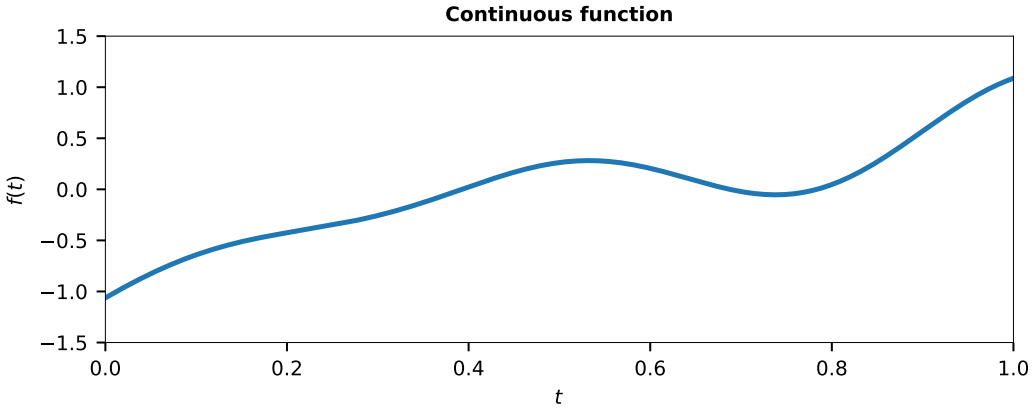


UNIVERSITY OF WATERLOO | FACULTY OF ENGINEERING

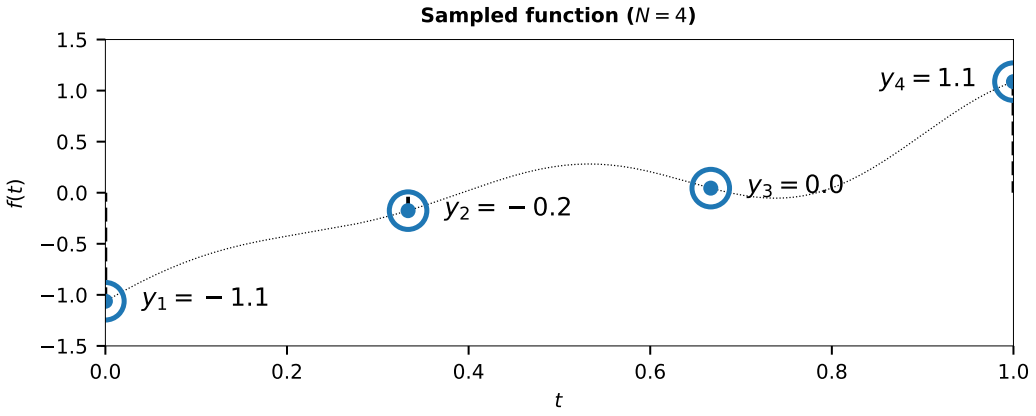
# Representing Stimulus Histories



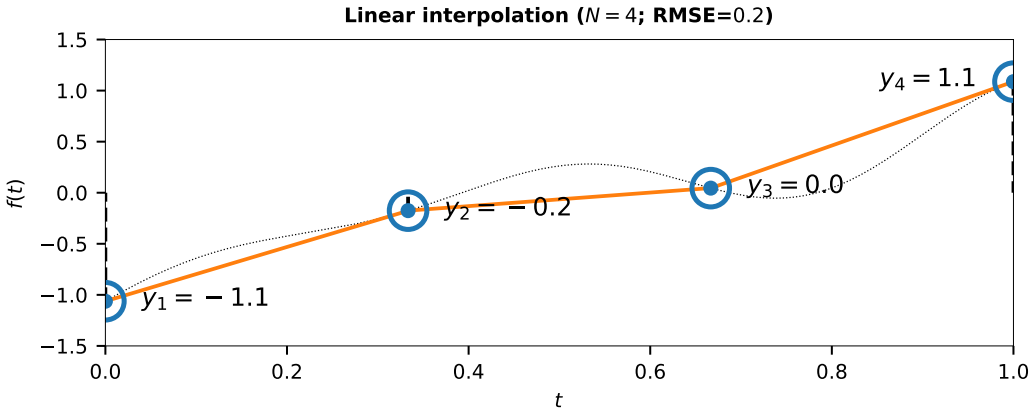
# Representing Functions: Sampling



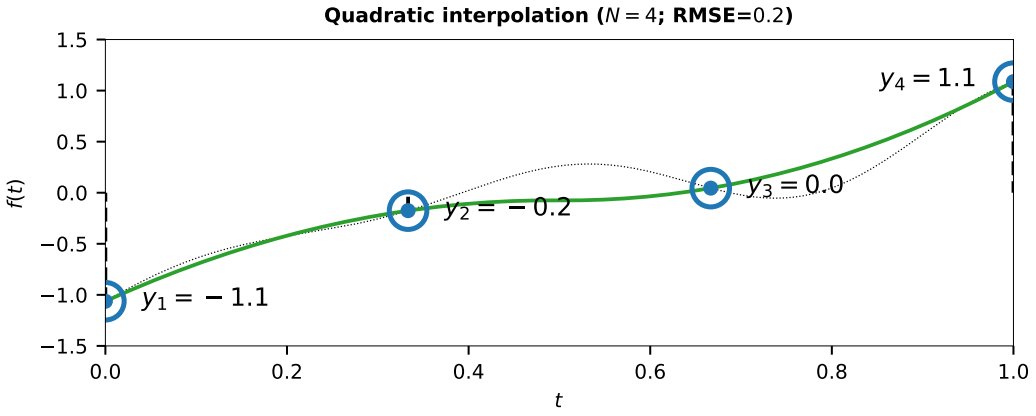
# Representing Functions: Sampling



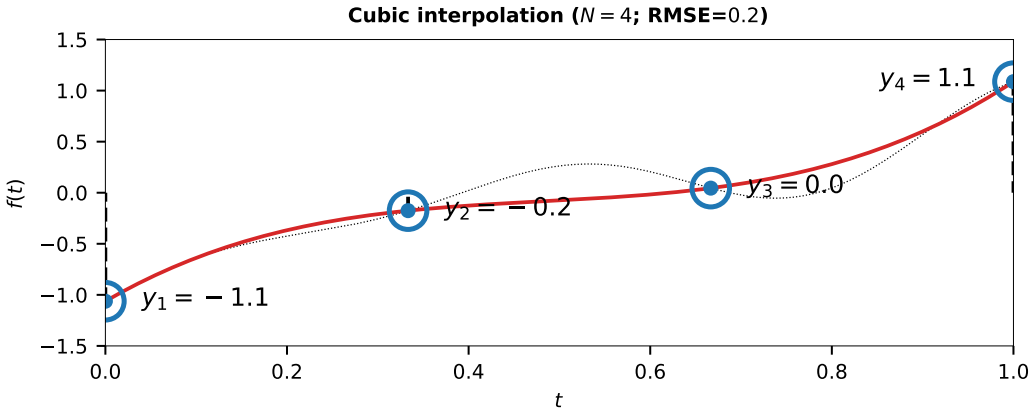
# Representing Functions: Sampling



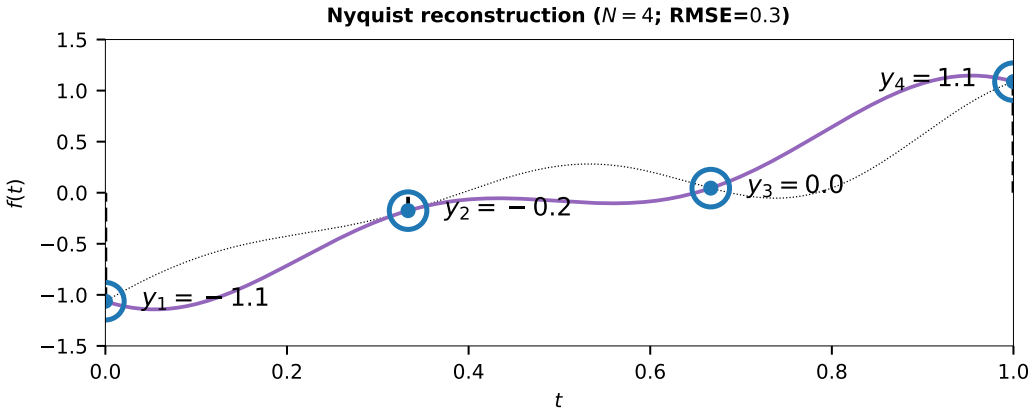
# Representing Functions: Sampling



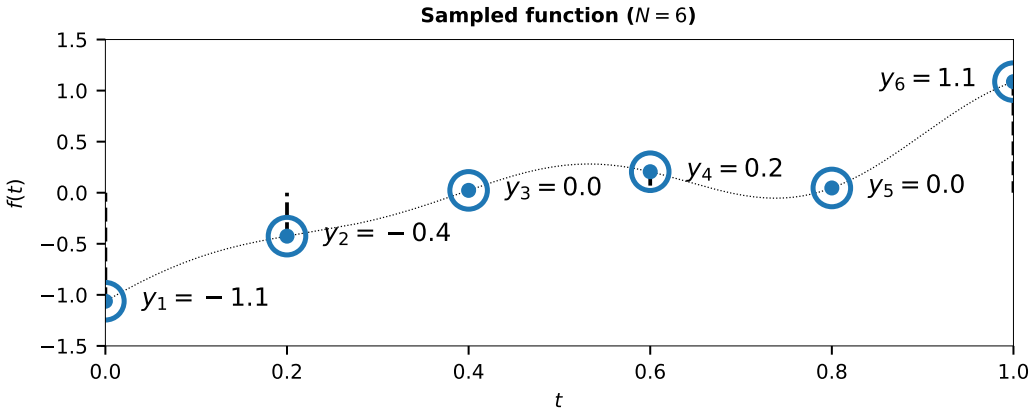
# Representing Functions: Sampling



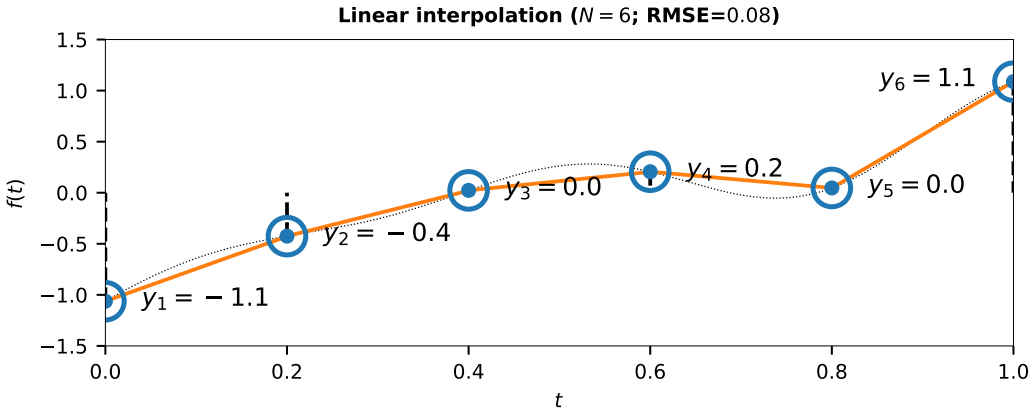
# Representing Functions: Sampling



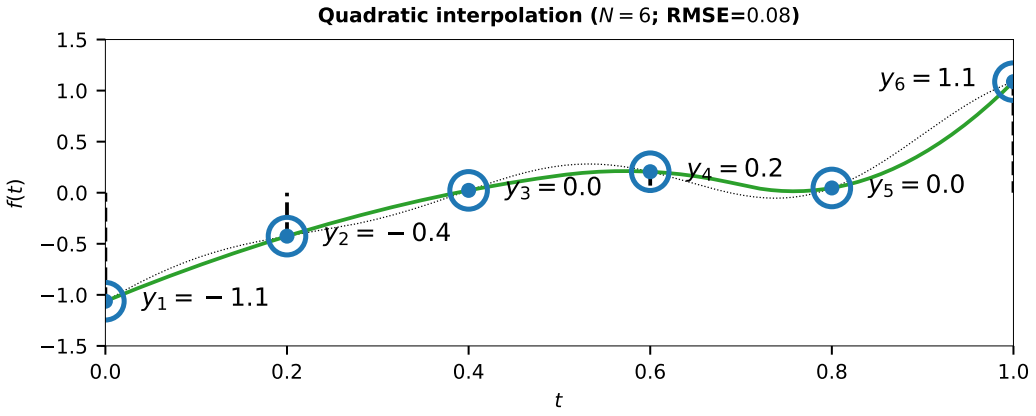
# Representing Functions: Sampling



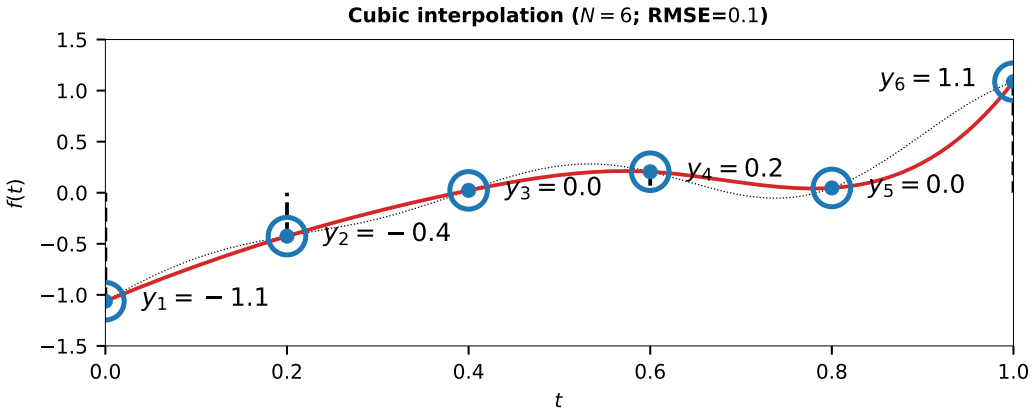
# Representing Functions: Sampling



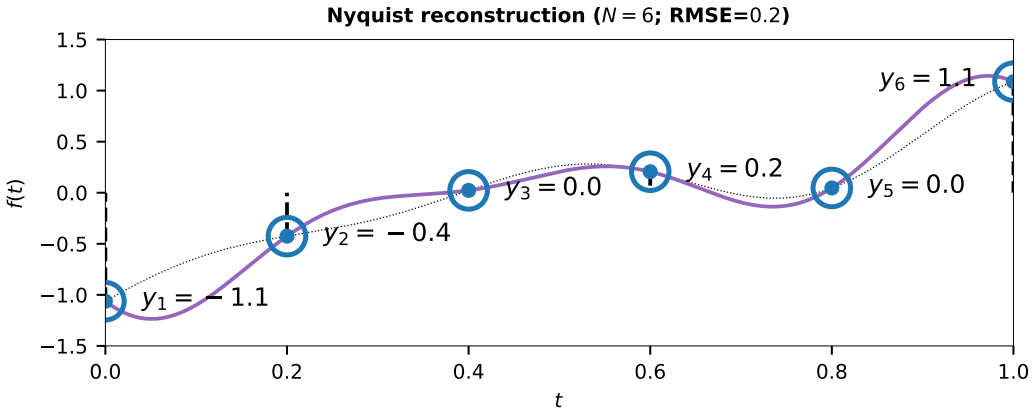
# Representing Functions: Sampling



# Representing Functions: Sampling



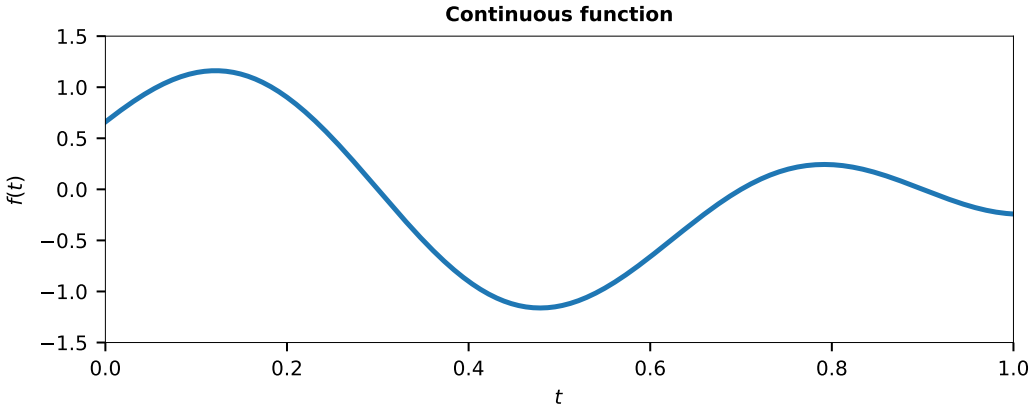
# Representing Functions: Sampling



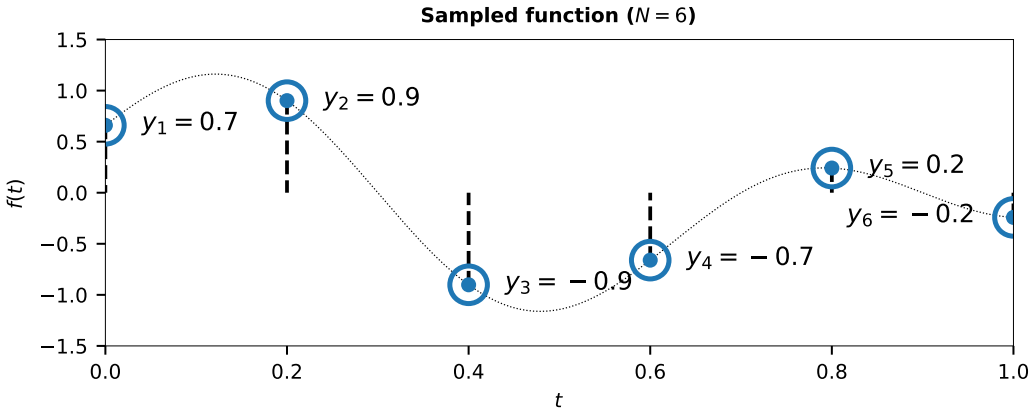
# Nyquist Sampling

*The Nyquist-Shannon Sampling Theorem*
If $f(t)$ contains no frequencies greater than $B$ then it is *completely* determined by samples spaced $\Delta t = \frac{1}{2B}$ apart ($N = 2BT$ equally spaced samples for a time-slice $[0, T)$). There is a *one-to-one mapping* between the samples $\mathbf{x}$ and the function $f(t)$.

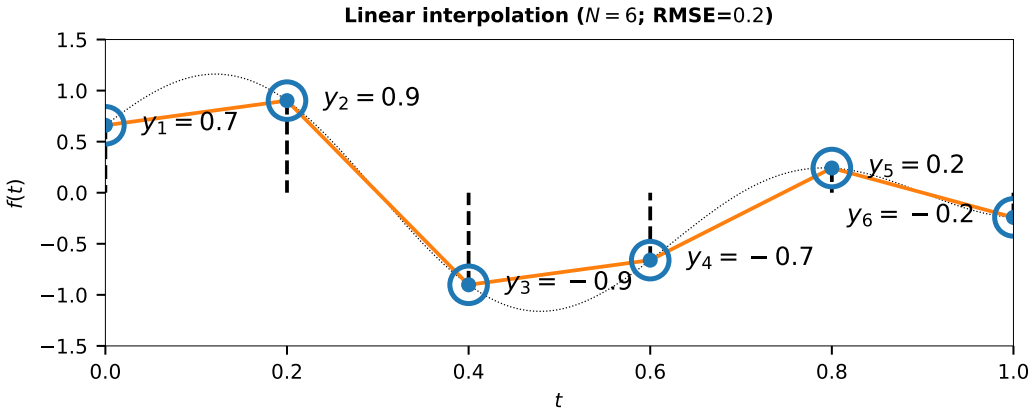
# Representing Functions: Sampling $<3$Hz



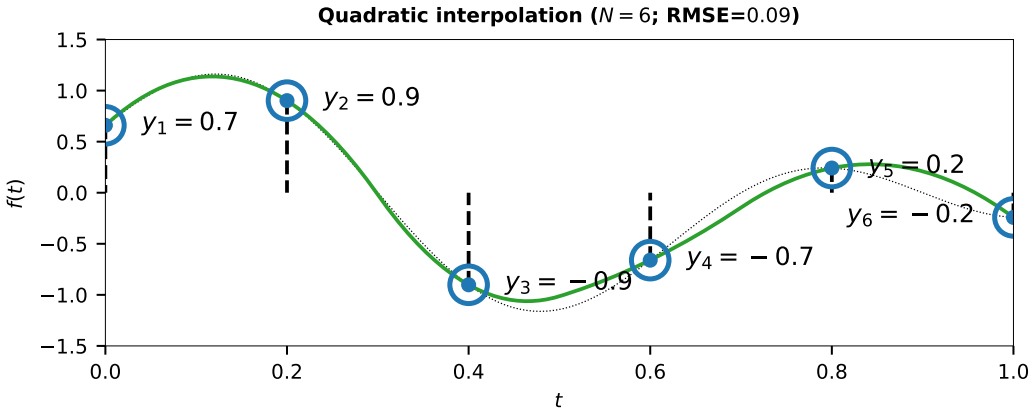
# Representing Functions: Sampling $<3$Hz



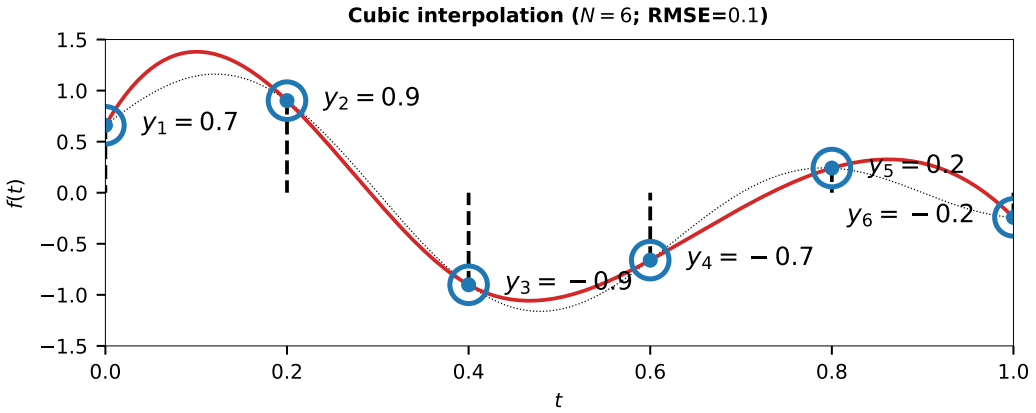
# Representing Functions: Sampling <3Hz



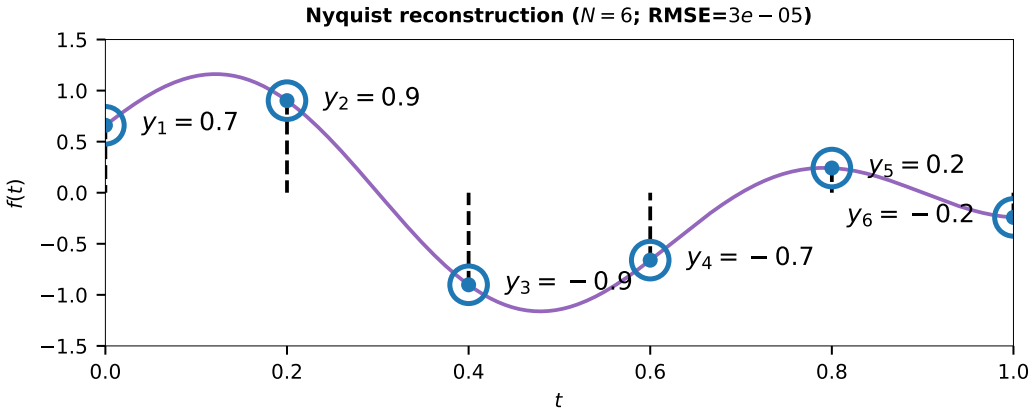
# Representing Functions: Sampling <3Hz



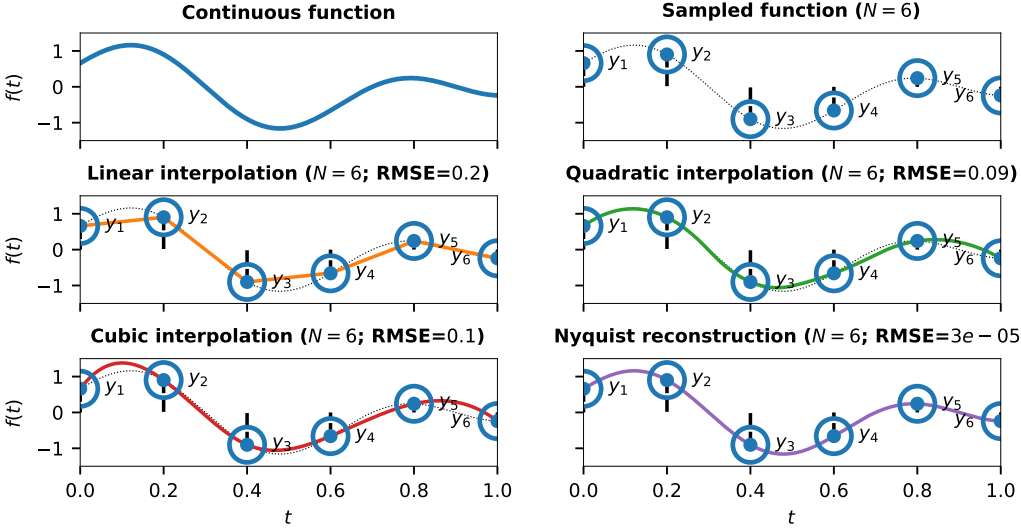
# Representing Functions: Sampling <3Hz



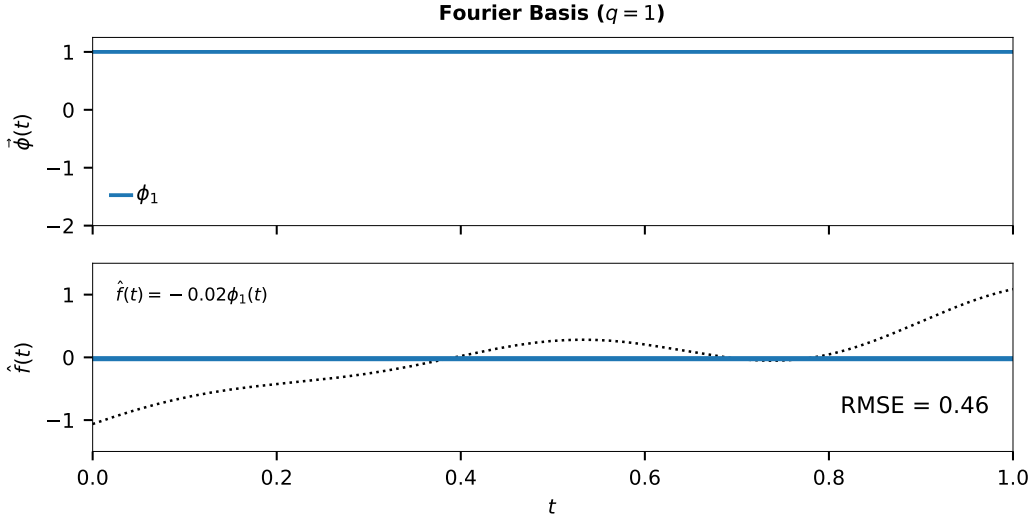
# Representing Functions: Sampling <3Hz



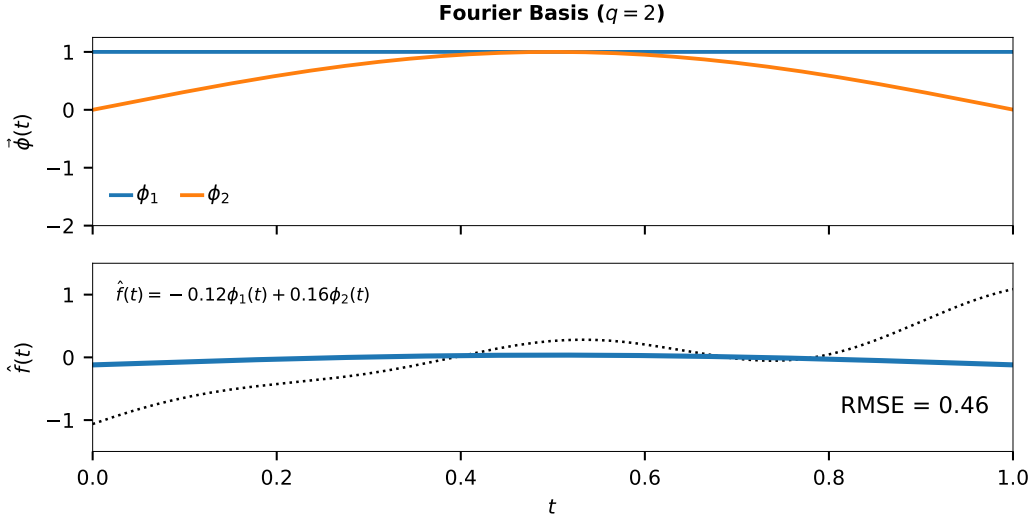
# Representing Functions: Sampling <3Hz



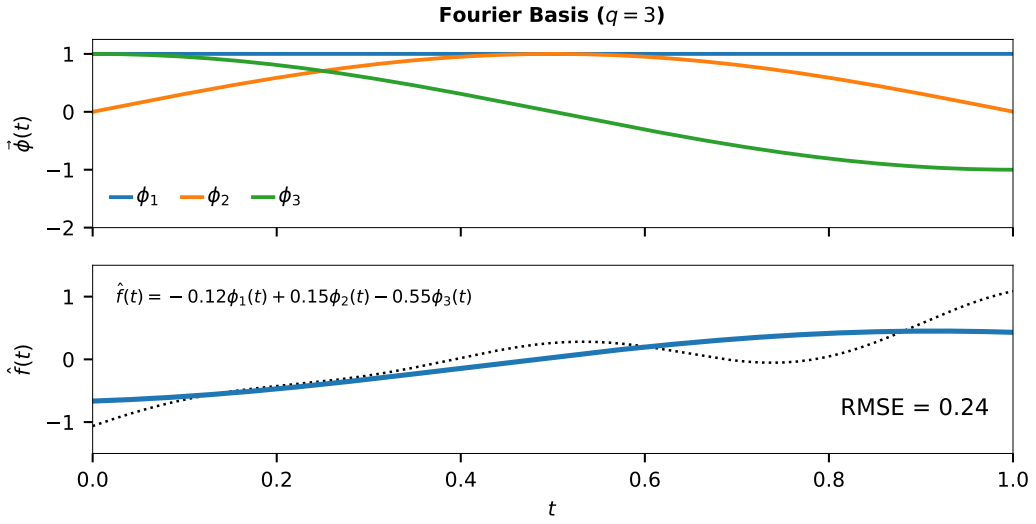
# Representing Functions: Fourier Basis



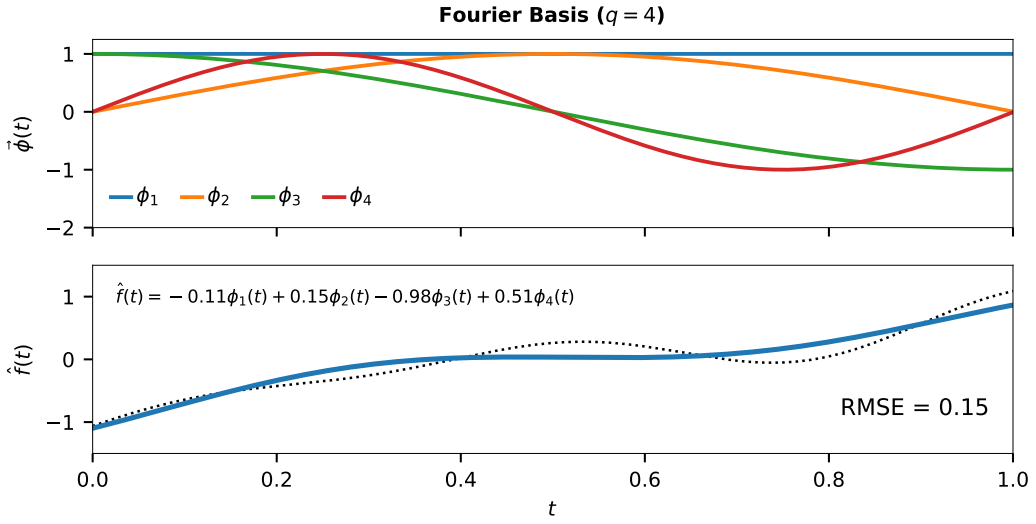
# Representing Functions: Fourier Basis



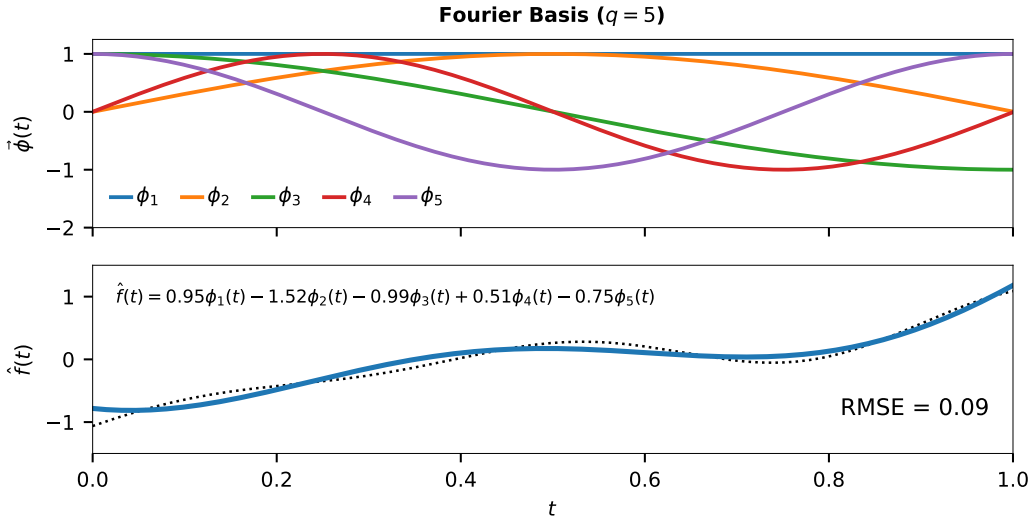
# Representing Functions: Fourier Basis



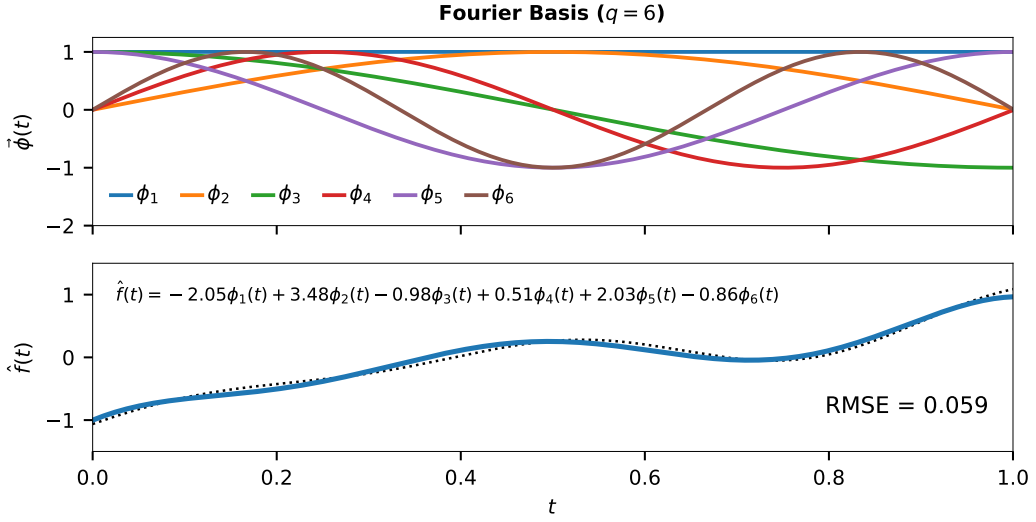
# Representing Functions: Fourier Basis



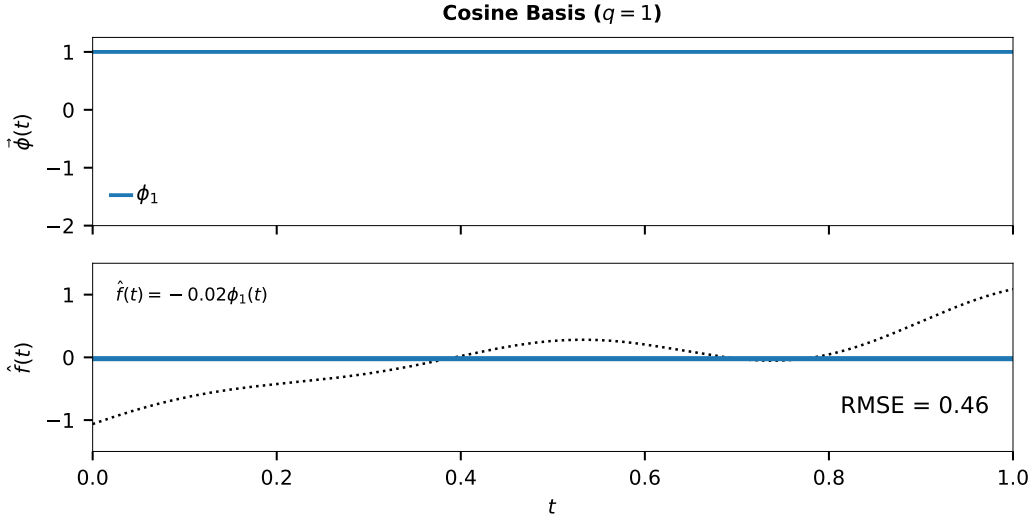
# Representing Functions: Fourier Basis



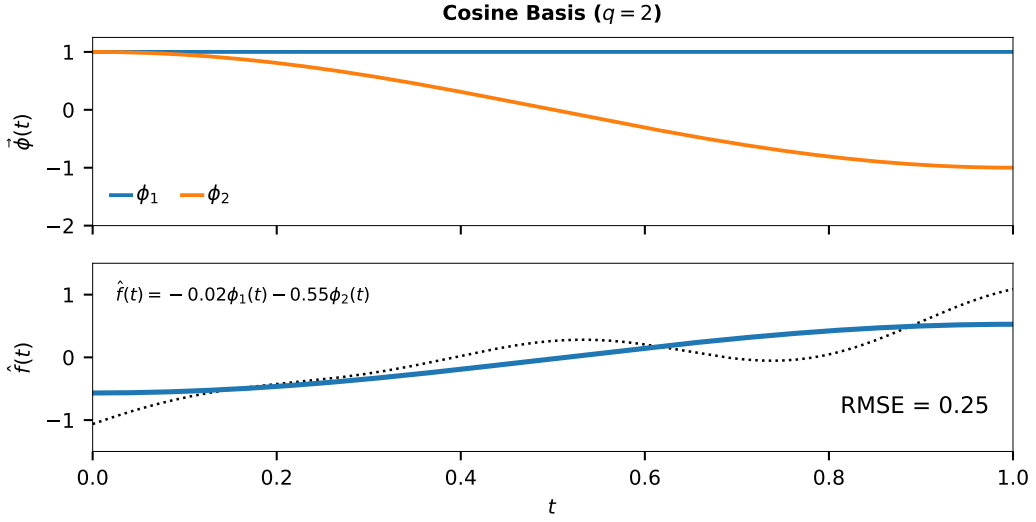
# Representing Functions: Fourier Basis



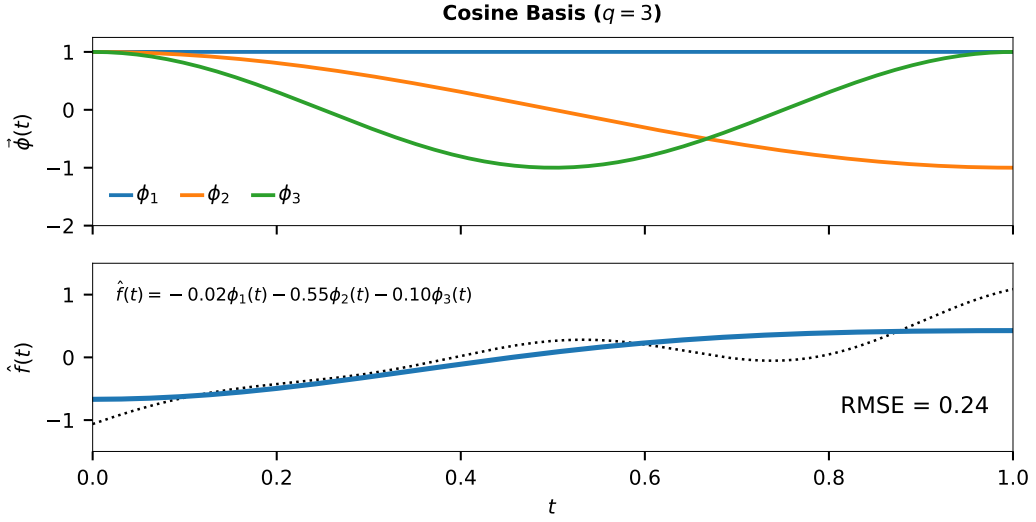
# Representing Functions: Cosine Basis



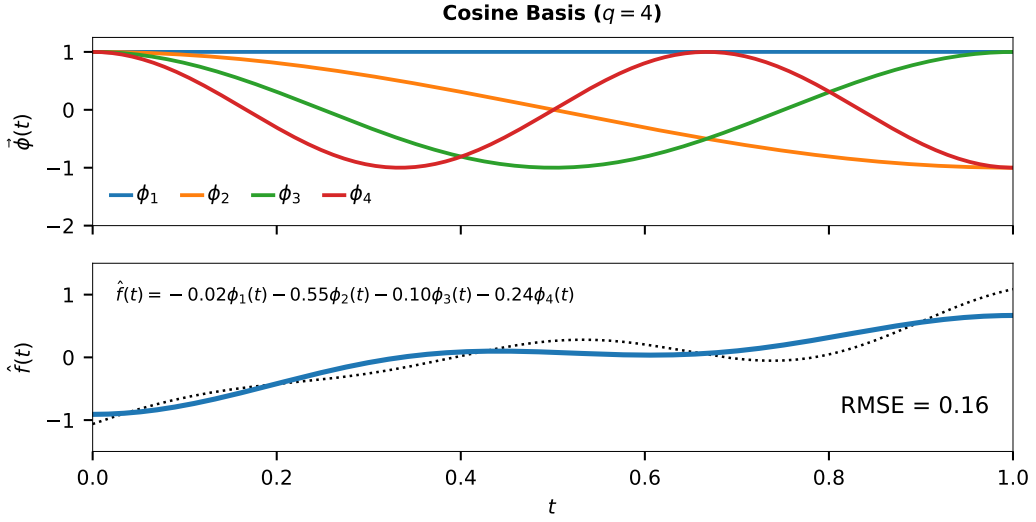
# Representing Functions: Cosine Basis



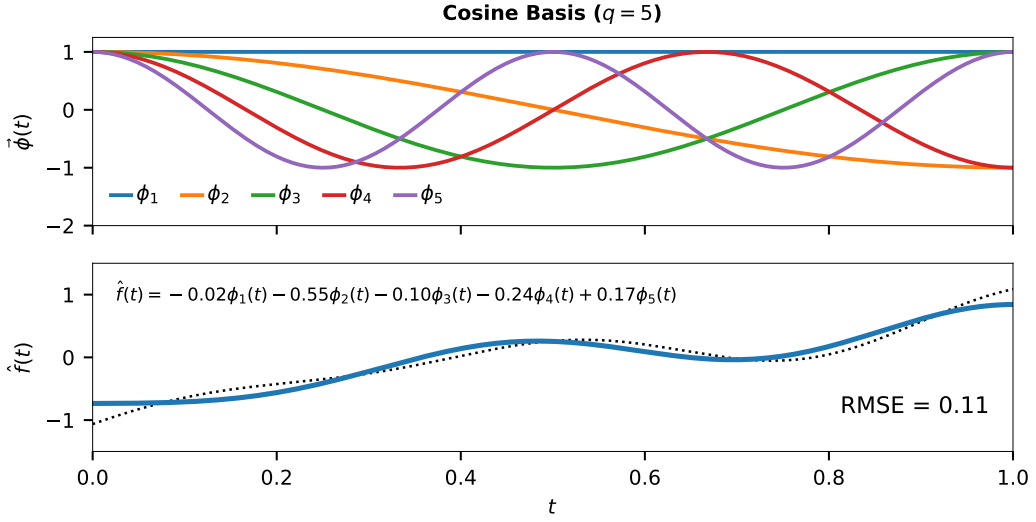
# Representing Functions: Cosine Basis



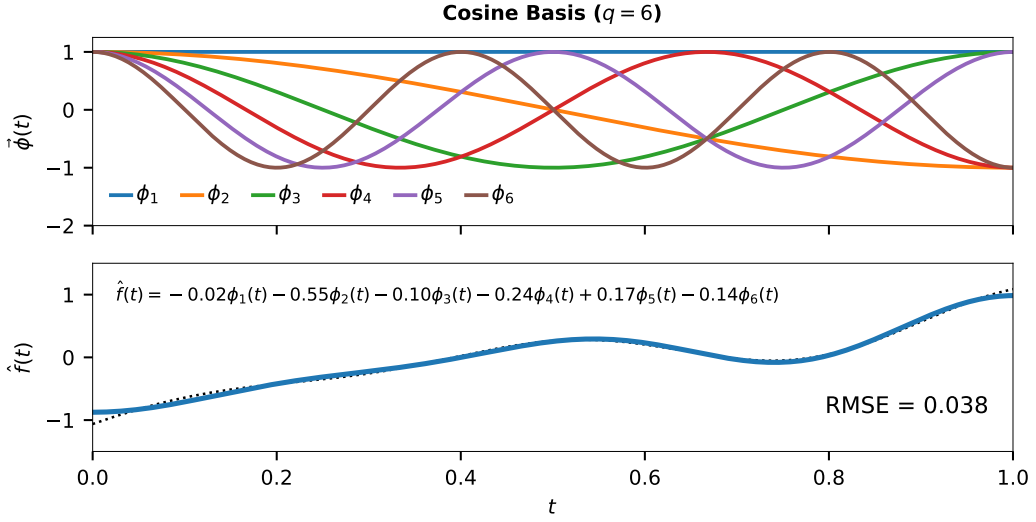
# Representing Functions: Cosine Basis



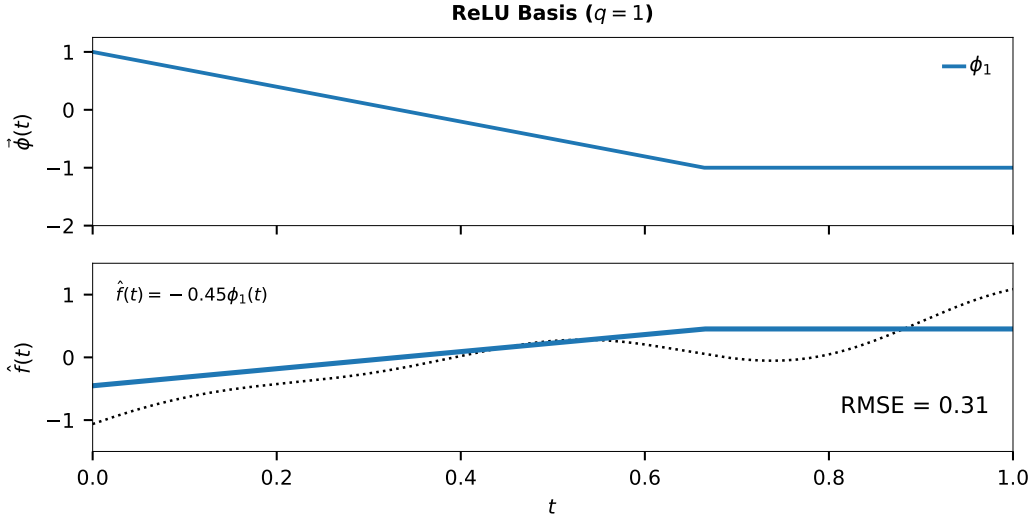
# Representing Functions: Cosine Basis



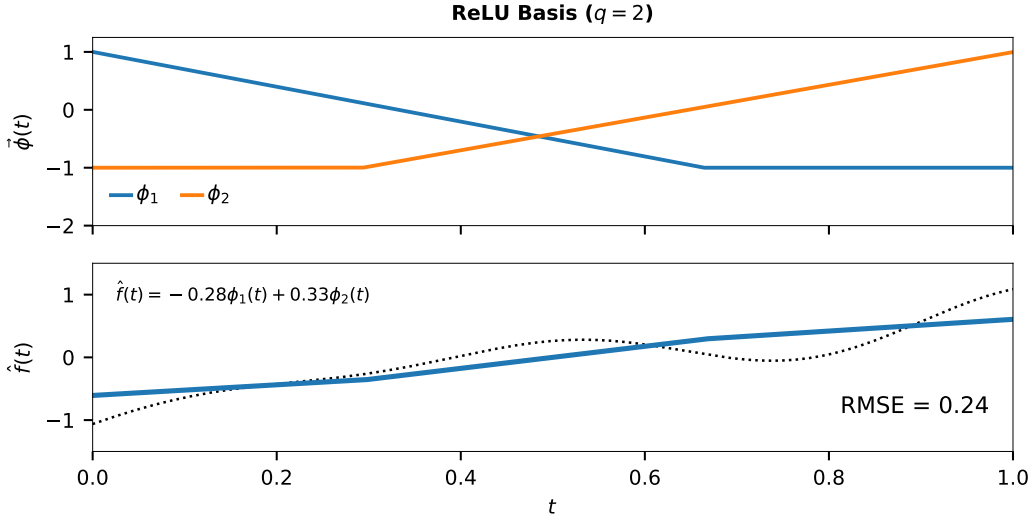
# Representing Functions: Cosine Basis



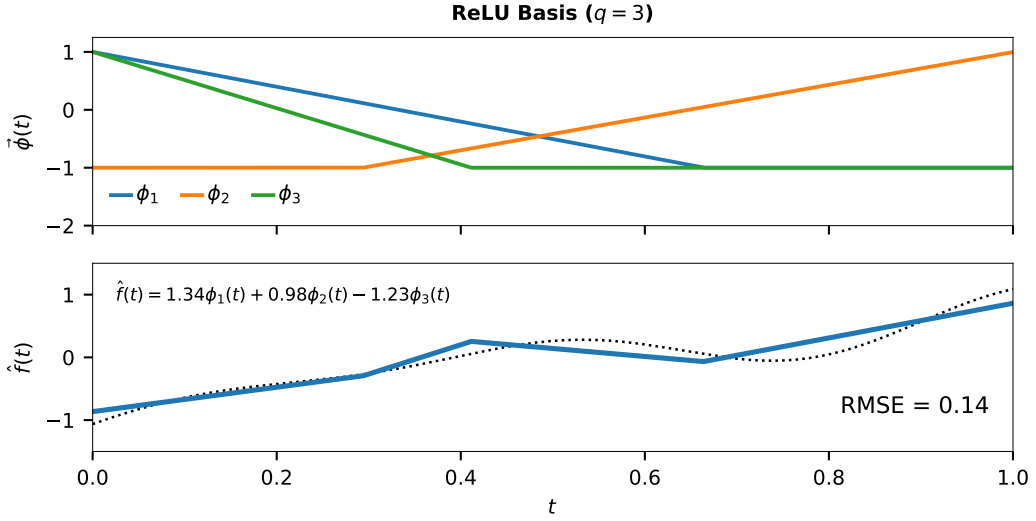
# Representing Functions: ReLU Basis



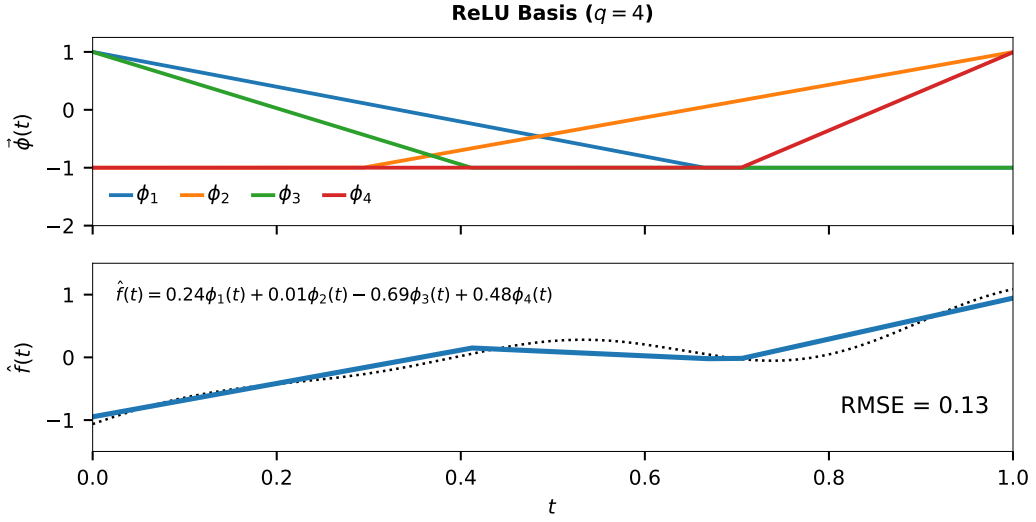
# Representing Functions: ReLU Basis



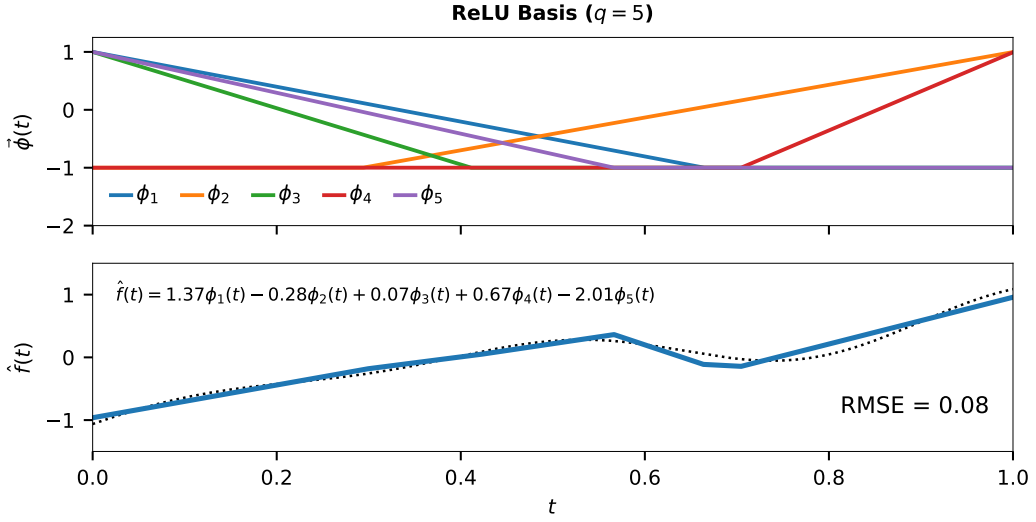
# Representing Functions: ReLU Basis



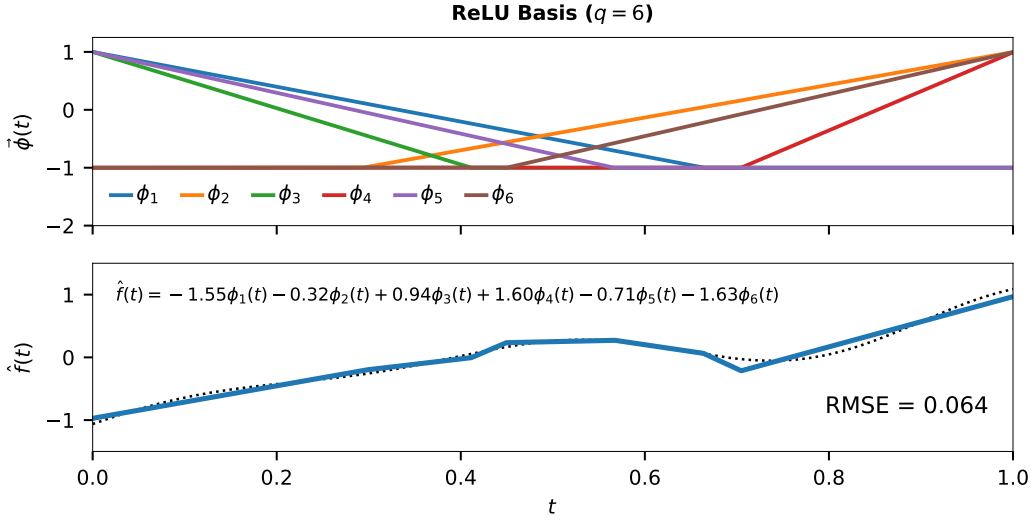
# Representing Functions: ReLU Basis



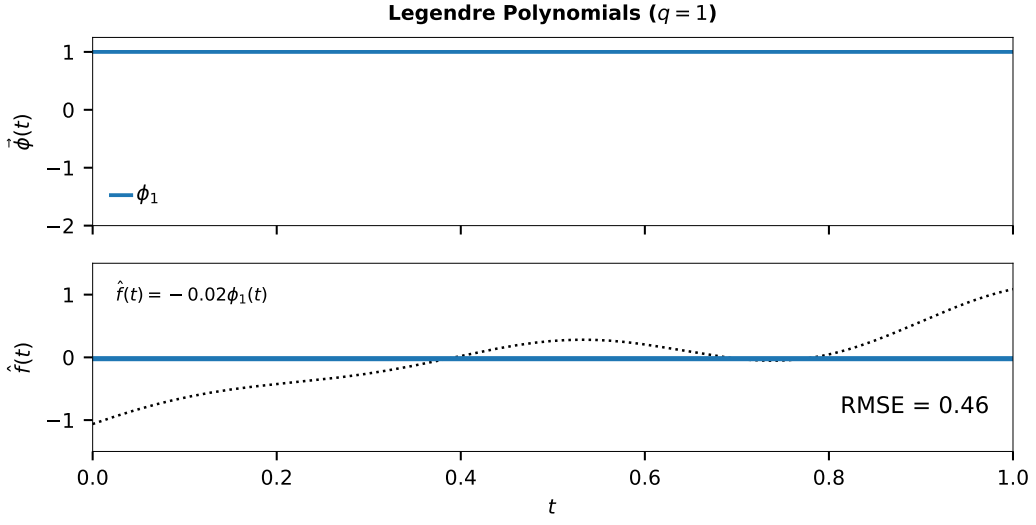
# Representing Functions: ReLU Basis



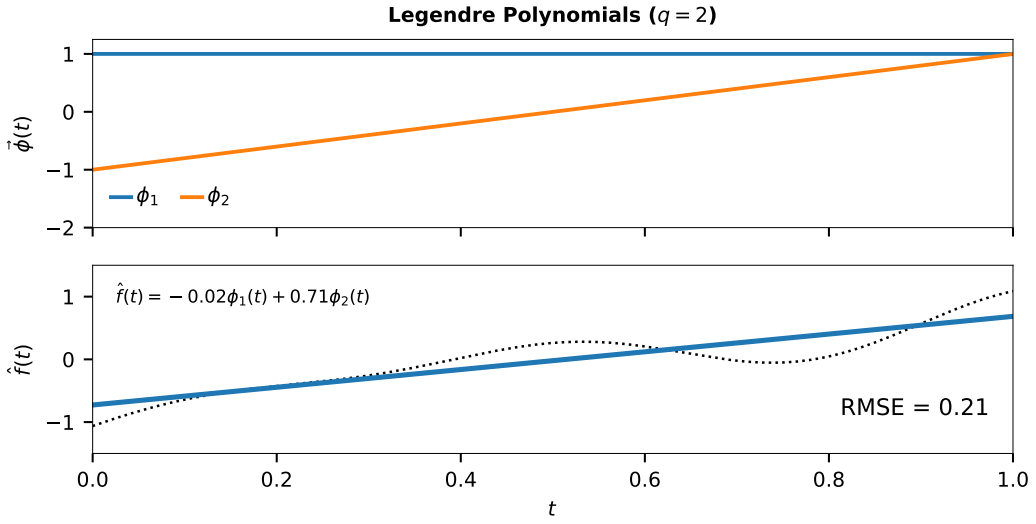
# Representing Functions: ReLU Basis



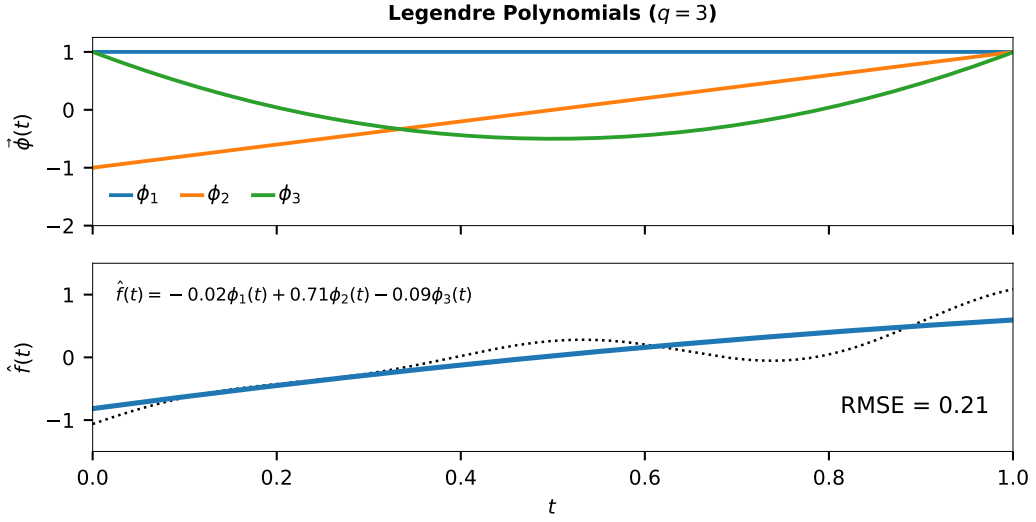
# Representing Functions: Legendre Basis



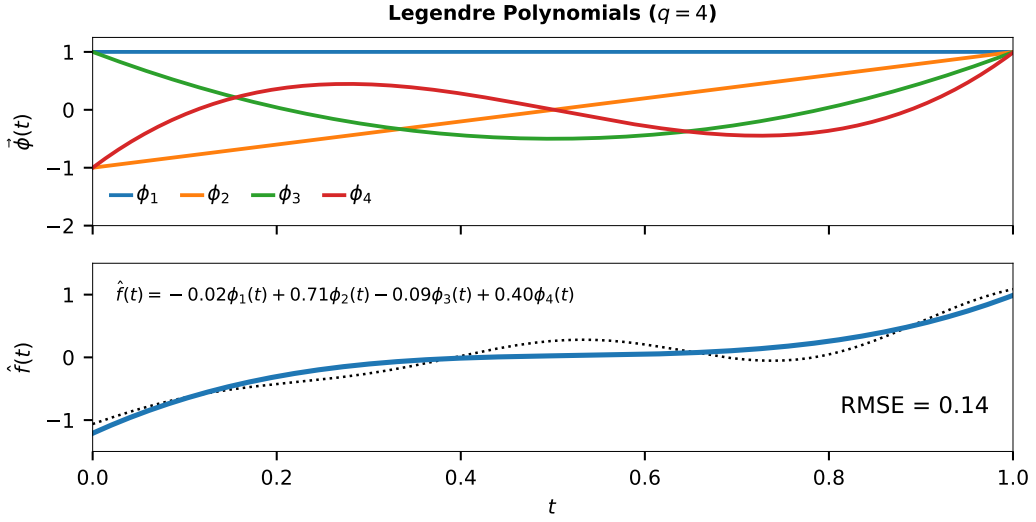
# Representing Functions: Legendre Basis



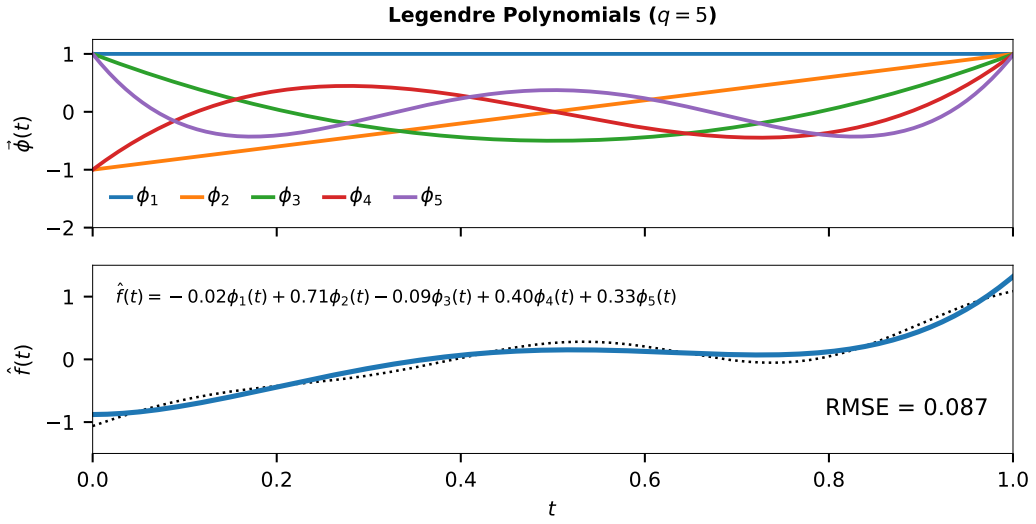
# Representing Functions: Legendre Basis



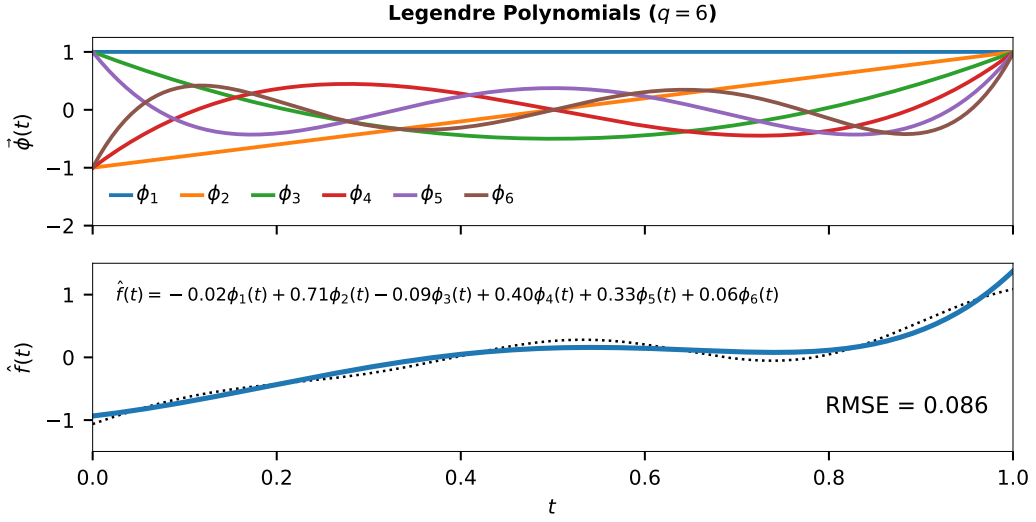
# Representing Functions: Legendre Basis



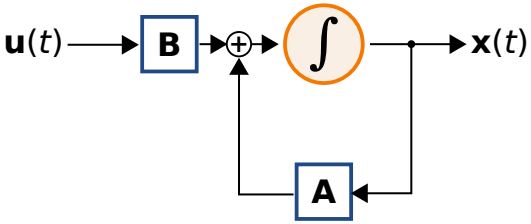
# Representing Functions: Legendre Basis

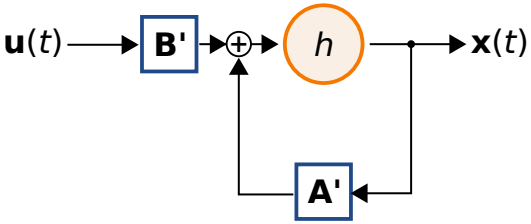


# Representing Functions: Legendre Basis



# Implementing the Delay Network



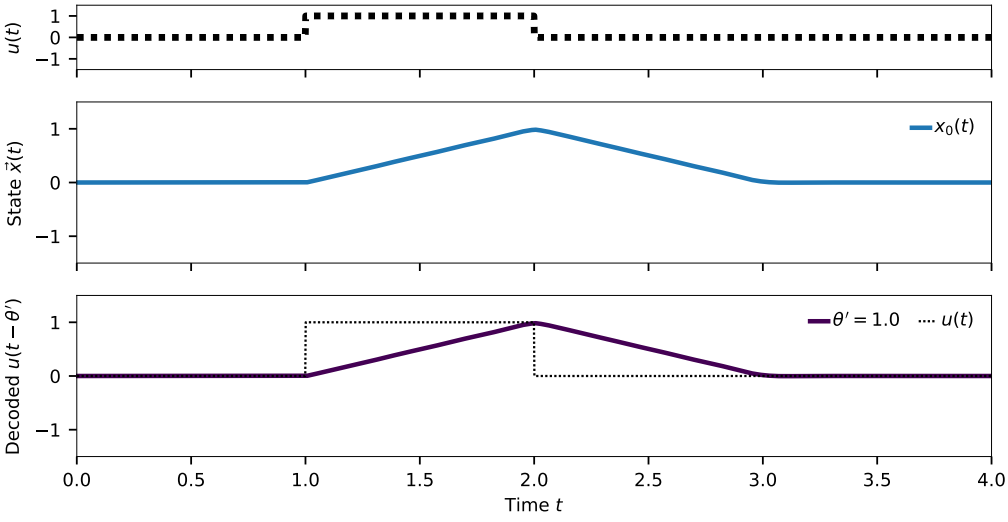




$$\theta\mathbf{A} = a_{ij} \in \mathbb{R}^{q \times q}, \qquad a_{ij} = \begin{cases} (2i+1)(-1) & i < j, \\ (2i+1)(-1)^{i-j+1} & i \geq j \end{cases}$$

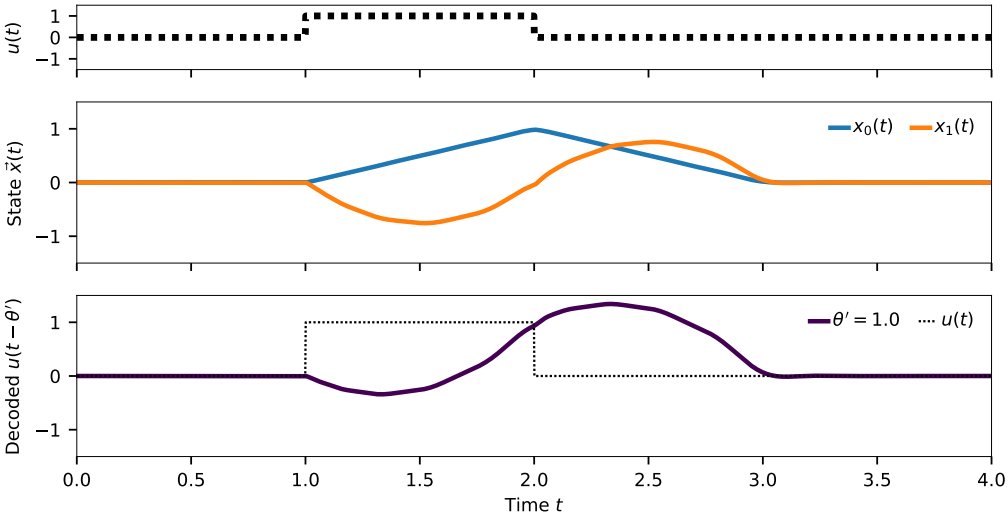
$$\theta\mathbf{B} = b_i \in \mathbb{R}^{q}, \qquad b_i = (2i+1)(-1)^i$$

$$\mathbf{A}' = \tau\mathbf{A} + \mathbf{I} \qquad \mathbf{B}' = \tau\mathbf{B}$$

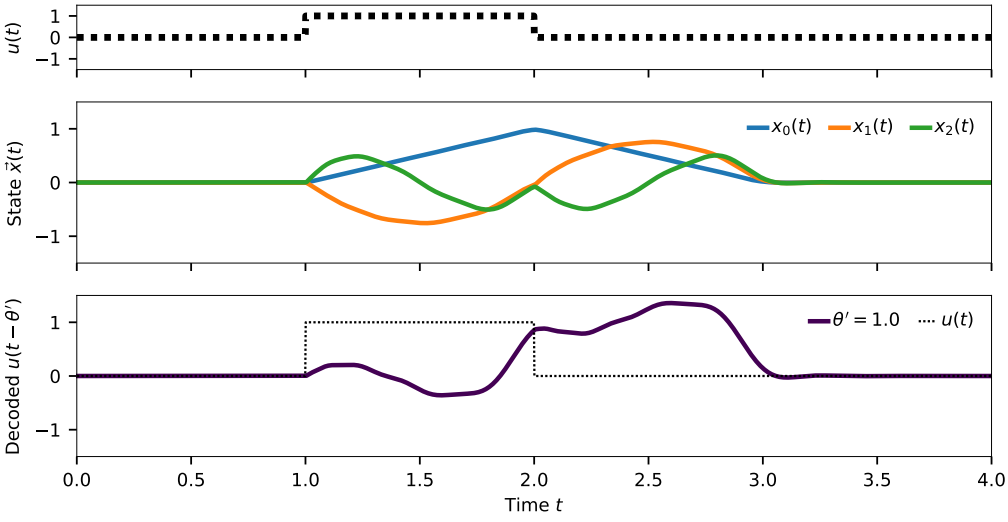
# Delay Network: Step Function



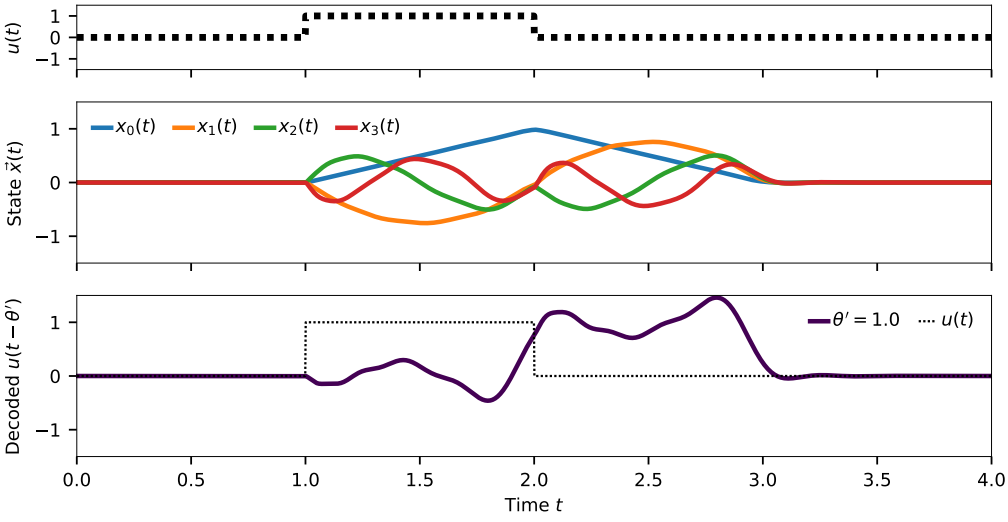
# Delay Network: Step Function



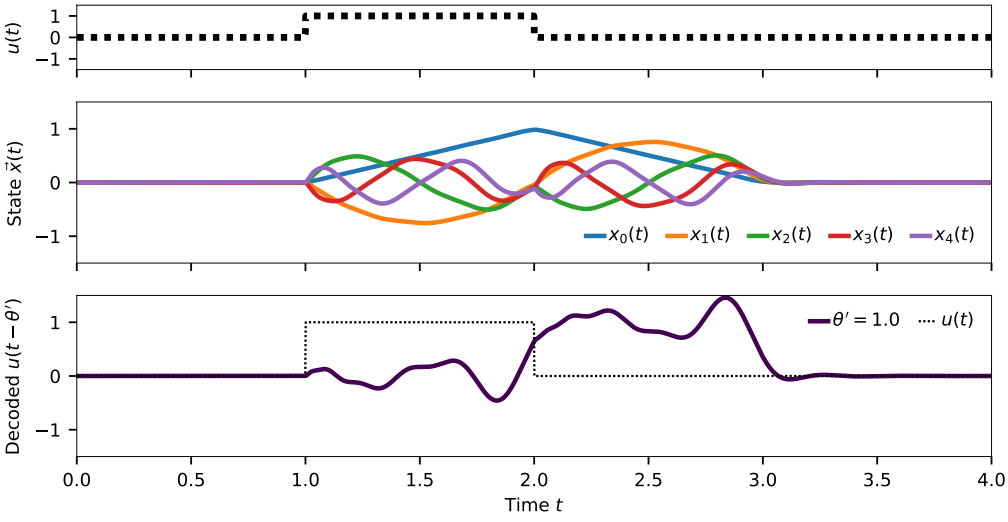
# Delay Network: Step Function



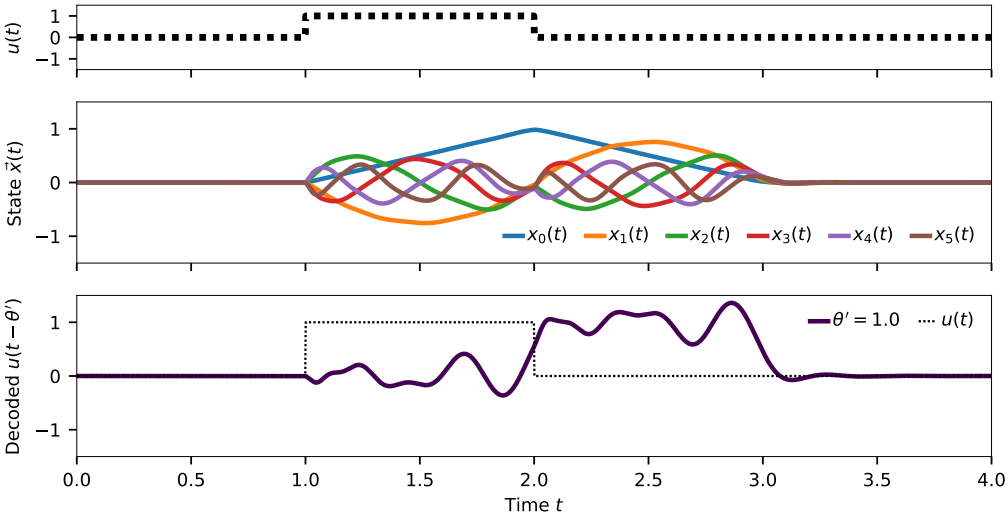
# Delay Network: Step Function



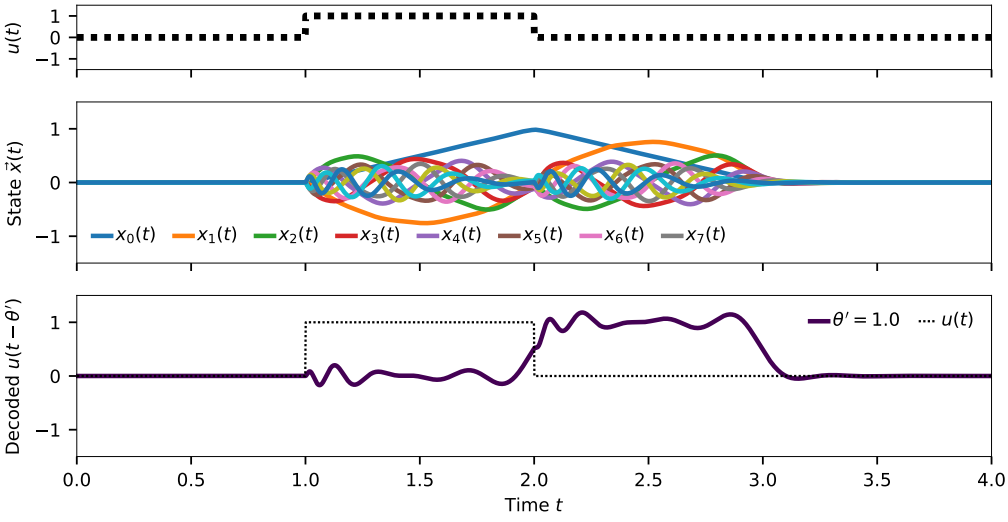
# Delay Network: Step Function



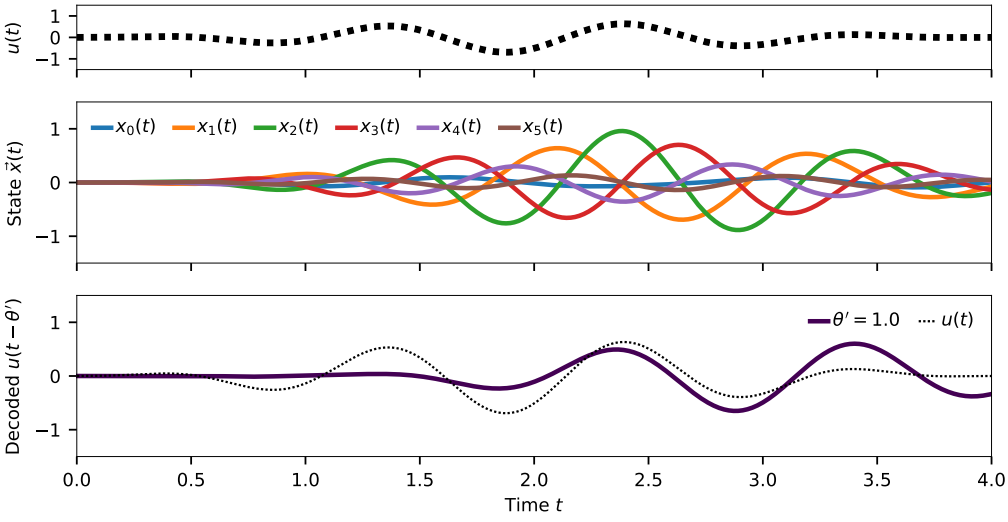
# Delay Network: Step Function



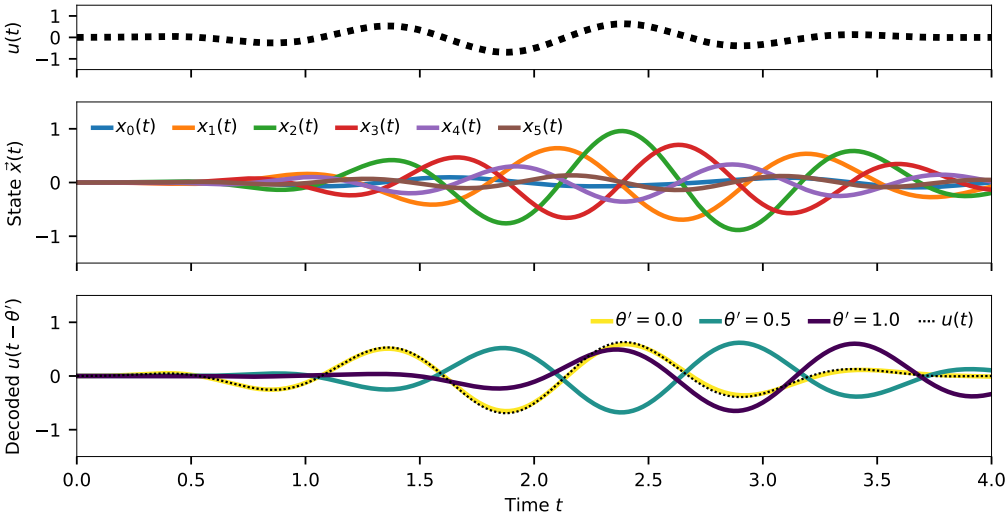
# Delay Network: Step Function



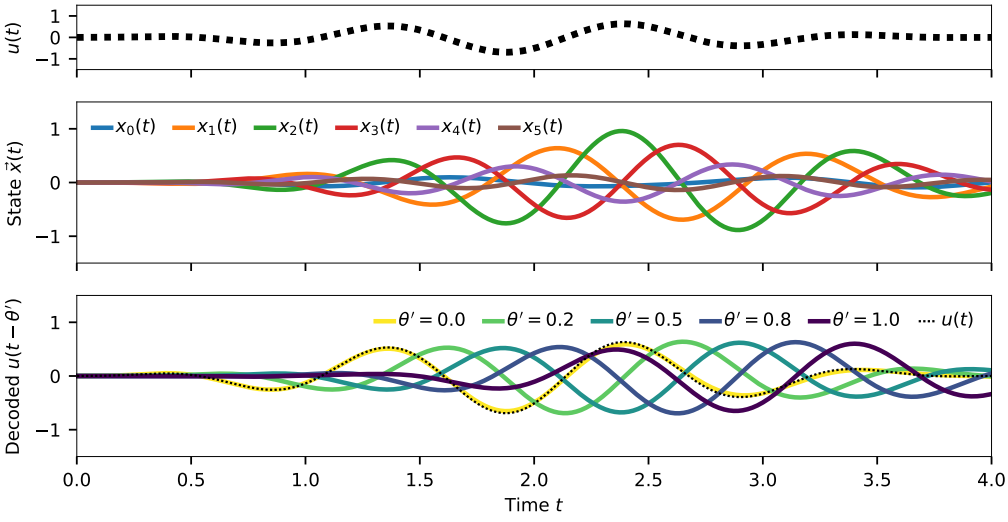
# Delay Network: Windowed Sine Function



# Delay Network: Windowed Sine Function

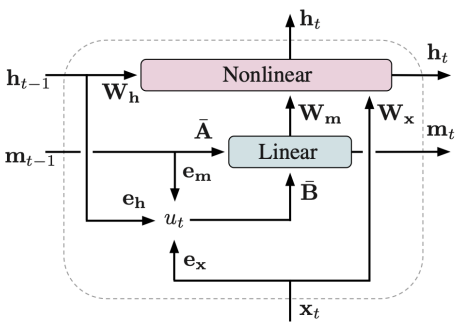


# Delay Network: Windowed Sine Function



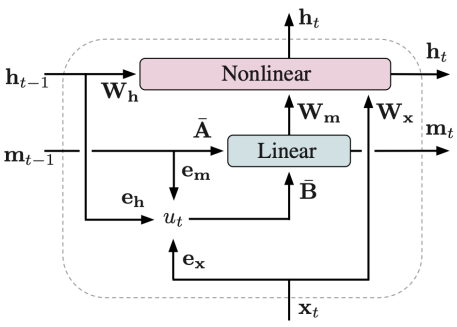
# From the Delay Network to the Legendre Memory Unit

- The *Legendre Memory Unit* (LMU) places the delay network's linear system at the core of a recurrent neural architecture



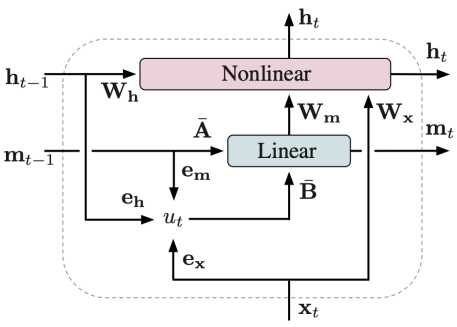
# From the Delay Network to the Legendre Memory Unit

- The *Legendre Memory Unit* (LMU) places the delay network's linear system at the core of a recurrent neural architecture
- Parameter-efficient: $\mathcal{O}(N)$ rather than the expected $\mathcal{O}(N^2)$



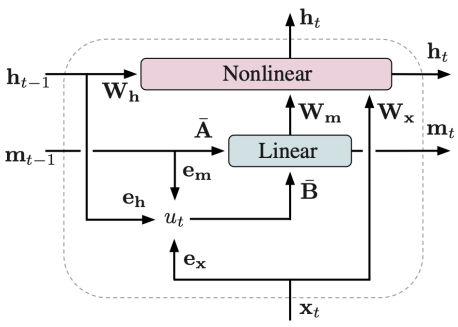
# From the Delay Network to the Legendre Memory Unit

- The *Legendre Memory Unit* (LMU) places the delay network's linear system at the core of a recurrent neural architecture
- Parameter-efficient: $\mathcal{O}(N)$ rather than the expected $\mathcal{O}(N^2)$
- Trainable feedforward, deployable recurrently



# From the Delay Network to the Legendre Memory Unit

- The *Legendre Memory Unit* (LMU) places the delay network's linear system at the core of a recurrent neural architecture
- Parameter-efficient: $\mathcal{O}(N)$ rather than the expected $\mathcal{O}(N^2)$
- Trainable feedforward, deployable recurrently
- Neural connection: explains time cells; runs dynamic and spiking (or not)



# State Space Models (SSMs)

- LSTMs (1995): long the dominant recurrent network, but training cannot be parallelized, and sequences beyond $\sim 10^3$ samples are out of reach

# State Space Models (SSMs)

- LSTMs (1995): long the dominant recurrent network, but training cannot be parallelized, and sequences beyond $\sim 10^3$ samples are out of reach
- Transformers (2017): parallel training, but compute and memory scale as $\mathcal{O}(N^2)$ in sequence length

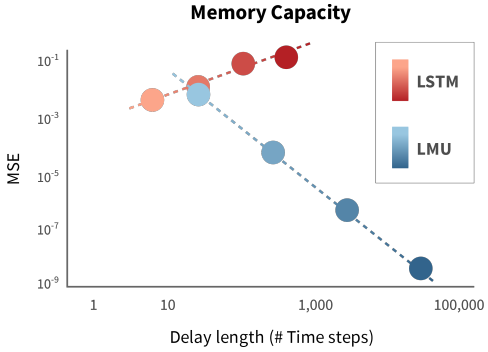
# State Space Models (SSMs)

- LSTMs (1995): long the dominant recurrent network, but training cannot be parallelized, and sequences beyond $\sim 10^3$ samples are out of reach
- Transformers (2017): parallel training, but compute and memory scale as $\mathcal{O}(N^2)$ in sequence length
- *SSMs*: the delay network/LMU (2018–2019, Waterloo) was the first; re-derived at Stanford as HiPPO (2020), followed by the S-series (2021) and Mamba (2023)

# State Space Models (SSMs)

- LSTMs (1995): long the dominant recurrent network, but training cannot be parallelized, and sequences beyond $\sim 10^3$ samples are out of reach
- Transformers (2017): parallel training, but compute and memory scale as $\mathcal{O}(N^2)$ in sequence length
- *SSMs*: the delay network/LMU (2018–2019, Waterloo) was the first; re-derived at Stanford as HiPPO (2020), followed by the S-series (2021) and Mamba (2023)
- SSMs now achieve state-of-the-art results on language, audio, and genomics, with SSM work at DeepMind, Meta, Nvidia, Princeton, Stanford, ...

# LMU Benchmarks




| Model | Accuracy |
|---|---|
| RNN-orth | 89.26 |
| RNN-id | 86.13 |
| LSTM | 89.86 |
| LSTM-chrono | 88.43 |
| GRU | 92.39 |
| JANET | 91.94 |
| SRU | 92.49 |
| GORU | 87.00 |
| NRU | 95.38 |
| Phased LSTM | 89.61 |
| LMU | 97.15 |
| HiPPO-LegS | 98.3 |
| FF-baseline | 98.20 |
| Our Model | **98.49** |

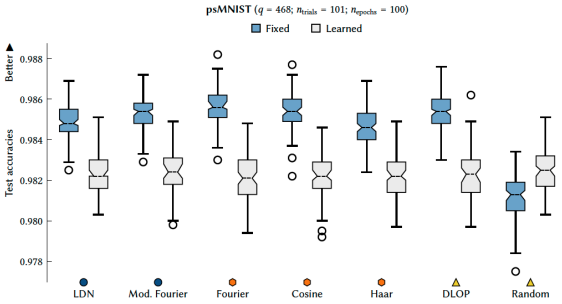
- Memory: $\sim 10^6\times$ lower error while remembering $\sim 10^4\times$ longer sequences than an LSTM
- psMNIST: (simplified) LMU set the state of the art (98.49 %) with 60 % fewer parameters
- Scaling: $\mathcal{O}(N)$ compute and $\mathcal{O}(1)$ memory, vs. $\mathcal{O}(N^2)$ for transformers

# SSMs in Practice (and Other Bases)

- Applications: LLMs (e.g. Mamba), speech recognition, keyword spotting, RF classification, anomaly detection, biosignals, nonlinear system prediction

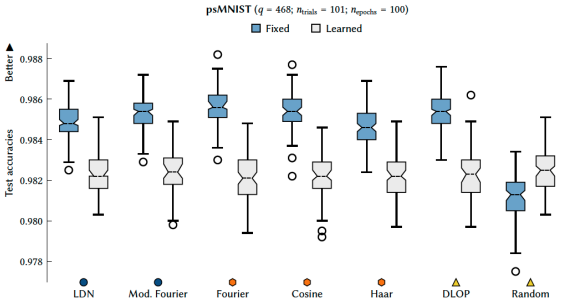
# SSMs in Practice (and Other Bases)

- Applications: LLMs (e.g. Mamba), speech recognition, keyword spotting, RF classification, anomaly detection, biosignals, nonlinear system prediction
- Other temporal bases work too (Fourier, cosine, Haar, ...); analytic bases often beat learned ones



# SSMs in Practice (and Other Bases)

- Applications: LLMs (e.g. Mamba), speech recognition, keyword spotting, RF classification, anomaly detection, biosignals, nonlinear system prediction
- Other temporal bases work too (Fourier, cosine, Haar, ...); analytic bases often beat learned ones




> "Next gen state space models [are] probably the next generation of transformers."
> — Jensen Huang, CEO Nvidia, 2024

# Image sources

**Title slide**
Infrared Photograph of a Sundial Near the Einstein Tower in Potsdam, Germany
Author: DrNRNowaczyk, 2007.
From Wikimedia.